[Insertar logo institucional]

Servicio Nacional de Aprendizaje — SENA
Centro de Electricidad, Electrónica y Telecomunicaciones — CEET
Tecnólogo en Análisis y Desarrollo de Software (ADSO) · Trimestre 6

CAPÍTULO I · MC-01

Modelo de Calidad del Producto de Software

Basado en ISO/IEC 25010:2023, ISO/IEC 25023 e ISO/IEC 25040

FASE: PLANEACIÓN

Proyecto formativo	EyeSchool — Sistema de control de asistencia escolar por QR y seguimiento académico
Ficha	[N.º de ficha]
Equipo / Autores	[Nombres completos de los aprendices]
Instructor(es)	[Nombre del instructor técnico]
Criterio de evaluación	Trimestre 6 — En el proyecto se evidencia algún modelo de calidad.
Versión del documento	0.3
Fecha	25/10/2026
Ciudad	Bogotá D.C.

Control del documento

Historial de versiones

Versión	Fecha	Autor(es)	Descripción del cambio
0.1	[dd/mm/aaaa]	[Autor]	Creación del documento a partir de la plantilla
0.2	25/10/2026	[Autor]	Diligenciamiento con los datos del proyecto: contexto, pesos, métricas adaptadas a la app móvil y línea base medida.
0.3	25/10/2026	[Autor]	Alcance ampliado a todo el sistema: API (FastAPI) y panel web (Next.js), con sus métricas, instrumentos y línea base.
[1.0]			Versión aprobada

Revisión y aprobación

Rol	Nombre	Firma	Fecha
Elaboró (aprendiz / equipo)			
Revisó (líder técnico / Scrum Master)			
Aprobó (instructor / Product Owner)			

Convenciones de esta plantilla

[Texto resaltado]	Campo que el equipo debe diligenciar. Reemplace el texto y elimine el resaltado.
Instrucción	Orientación para diligenciar la sección. Elimine estos recuadros en la versión final.
DATOS DE EJEMPLO	Gráficas ilustrativas. Reemplácelas por las generadas con los datos reales del proyecto.

Ubicación en el paquete documental

Documento que lo precede	Ninguno — es el documento rector del paquete
Documento que lo sucede	MC-02 Matriz de verificación de calidad
Norma / referente principal	ISO/IEC 25010:2023 · ISO/IEC 25023:2016 · ISO/IEC 25040:2024 · ISO/IEC/IEEE 12207:2017

Contenido

1. Introducción 4
1.1 Propósito 4
1.2 Alcance 4
1.3 Diferencia entre calidad de producto y calidad de proceso 4
1.4 Referencias normativas 4
2. Contexto del sistema 5
3. Modelo de calidad adoptado: ISO/IEC 25010:2023 5
3.1 Priorización de características 6
4. Especificación de métricas y umbrales 7
5. Modelo de evaluación y agregación 10
6. Proceso de evaluación (ISO/IEC 25040) y puntos de control 10
7. Roles y responsabilidades de calidad 11
8. Definición de Terminado (DoD) alineada al modelo 11
9. Control de cambios del modelo 12
10. Firmas 12

Si la tabla de contenido aparece vacía: clic derecho sobre ella → Actualizar campo → Actualizar toda la tabla.

1. Introducción

1.1 Propósito

Este documento define el modelo de calidad que el equipo adopta para el sistema EyeSchool: las características de calidad priorizadas, las métricas con que se medirán, los umbrales de aceptación y los puntos de control (quality gates) a lo largo del ciclo de vida. Es el documento rector del paquete: los planes de pruebas, el análisis estático, los planes de despliegue y el informe de liberación se evalúan contra lo aquí establecido.

1.2 Alcance

Aplica a los tres componentes del sistema EyeSchool y a su base de datos: la aplicación móvil (Flutter, Android) versión 1.0.1, la API REST (FastAPI) versión 0.1.0 con su base PostgreSQL, y el panel web (Next.js) versión 0.1.0. Cada componente vive en su propio repositorio y tiene su propio pipeline de CI; las métricas de la Tabla 4 indican a qué componente aplican.

1.3 Diferencia entre calidad de producto y calidad de proceso

Tabla 1. *Dimensiones de calidad consideradas*

Dimensión	Pregunta que responde	Referente normativo	Evidencia en el proyecto
Calidad del producto	¿El software cumple características medibles de calidad?	ISO/IEC 25010 (modelo) · ISO/IEC 25023 (métricas)	Este documento, MC-02, informes PR-03 a PR-09
Calidad en uso	¿El usuario logra sus objetivos con eficacia, eficiencia y satisfacción?	ISO/IEC 25019:2023	PR-08 pruebas de aceptación, encuestas de satisfacción
Calidad del proceso	¿El ciclo de vida se ejecuta de forma controlada y repetible?	ISO/IEC/IEEE 12207 · Scrum Guide	GC-01, tablero Scrum, actas de sprint, workflows de GitHub Actions
Proceso de evaluación	¿Cómo se evalúa la calidad de forma sistemática?	ISO/IEC 25040	Sección 6 de este documento

1.4 Referencias normativas

Tabla 2. *Normas y referentes*

Referencia	Tipo	Uso en el proyecto
ISO/IEC 25010:2023 — Product quality model	Norma	Modelo de 9 características y subcaracterísticas de calidad del producto.
ISO/IEC 25023:2016 — Measurement of system and software product quality	Norma	Catálogo de métricas para cuantificar cada subcaracterística.
ISO/IEC 25040 — Quality evaluation framework	Norma	Proceso de evaluación: establecer requisitos, especificar, diseñar, ejecutar y concluir.

Referencia	Tipo	Uso en el proyecto
ISO/IEC/IEEE 12207:2017 — Software life cycle processes	Norma	Marco de procesos del ciclo de vida y procesos de soporte.
ISO/IEC/IEEE 29119 (partes 1–3)	Norma	Conceptos, procesos y documentación de pruebas.
Glosario ISTQB (CTFL v4.0)	Referente	Terminología estándar de pruebas. No es una norma, es un esquema de certificación.
OWASP Top 10 · OWASP API Security Top 10 · OWASP MASVS	Referente	Requisitos y verificación de seguridad del panel web, la API y la app móvil.
WCAG 2.2	Referente	Criterios de contraste y de etiquetas para la característica capacidad de interacción.

2. Contexto del sistema

Nombre del sistema	EyeSchool
Objetivo de negocio	Reemplazar el llamado a lista en papel por un registro de asistencia con código QR, rápido y confiable incluso sin conexión, y dar a estudiantes y acudientes acceso inmediato a asistencia, notas, horarios y novedades para detectar a tiempo el ausentismo y el bajo rendimiento.
Usuarios y roles	Administrador (usuarios, asistencia masiva, novedades) · Docente (escaneo QR, asistencia por clase, notas) · Estudiante (QR personal; consulta de notas, horario, asistencia y novedades) · Padre/Acudiente (seguimiento de asistencia y notas de sus hijos, boletín PDF). Los cuatro roles usan la app móvil y el panel web; crear y editar usuarios solo se hace en el panel web.
Componentes y stack	Móvil: Flutter (Dart 3.12) con Riverpod, go_router y Dio · Web: Next.js 16 con React 19 y TypeScript · API: FastAPI (Python 3.11, SQLAlchemy, PyJWT) en Azure App Service · BD: PostgreSQL · Sesiones: Redis · Notificaciones: Firebase Cloud Messaging · CI/CD: GitHub Actions (un pipeline por repositorio) y GitHub Releases para la actualización OTA de la app.
Criticidad del sistema	Media-Alta. El sistema trata datos personales de menores de edad y la asistencia es un registro oficial que los acudientes consultan para saber si el estudiante llegó. Una falla no causa daño físico ni pérdidas económicas directas.
Restricciones	Ley 1581 de 2012 y Decreto 1377 de 2013 (datos de niños, niñas y adolescentes, con autorización del representante legal) · Duración del trimestre · Planes gratuitos o educativos de Azure y Firebase · Android como plataforma principal · Celulares de gama media/baja y conectividad intermitente en el colegio.

3. Modelo de calidad adoptado: ISO/IEC 25010:2023

La norma ISO/IEC 25010:2023 define la calidad del producto mediante **nueve características**, cada una descompuesta en subcaracterísticas medibles. La versión 2023 incorpora la característica **protección (safety)**, renombra la usabilidad como **capacidad de interacción** y la portabilidad como **flexibilidad**.

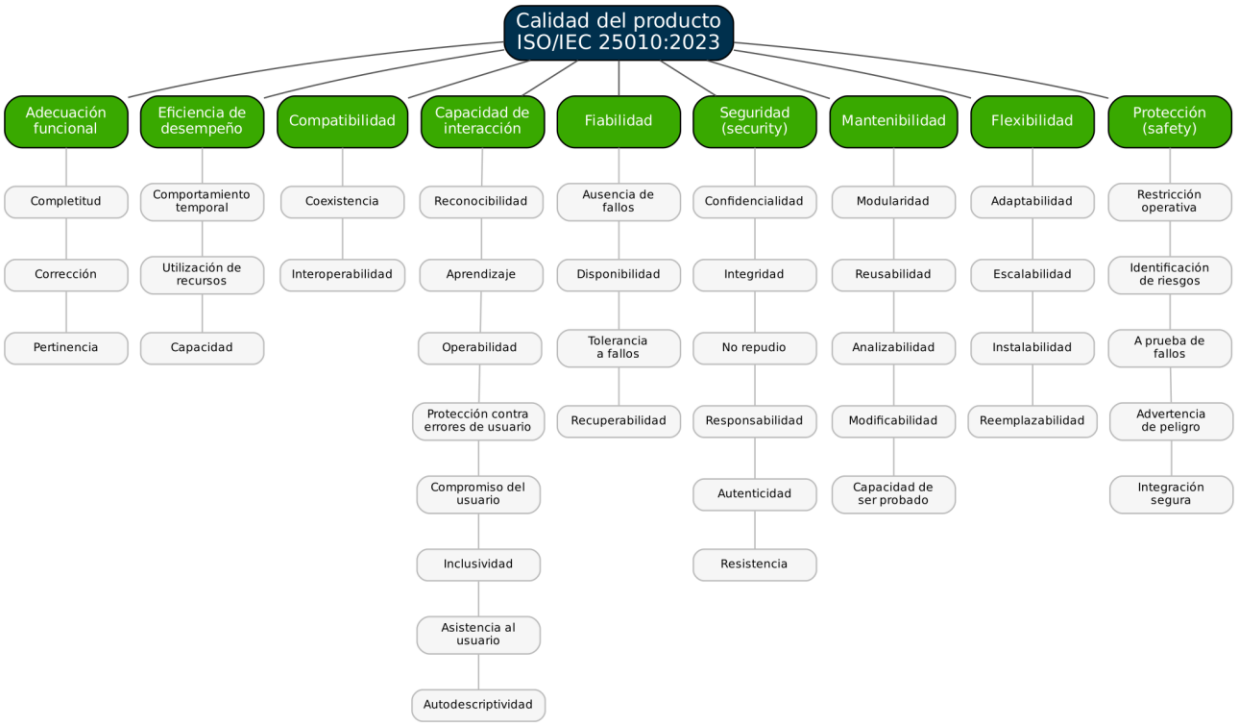


Figura 1. *Modelo de calidad del producto ISO/IEC 25010:2023* Fuente: Adaptado de ISO/IEC 25010:2023

3.1 Priorización de características

Cada característica recibe un peso de 0 a 5 según la criticidad del sistema. Las características con peso ≥ 4 tienen al menos dos métricas en la Tabla 4. La suma de los pesos (33) es el denominador del índice global de calidad de la sección 5.

Tabla 3. *Priorización de características de calidad*

Característica	Peso (0–5)	Justificación	Prioridad
Adecuación funcional	5	Es el propósito del sistema: registrar asistencia por QR y publicar notas, horarios y novedades comprometidos en el backlog.	Alta
Eficiencia de desempeño	3	El docente escanea a todo el grupo en la entrada y la API atiende a la vez la app y la web; deben responder rápido en celulares de gama media/baja y en horas pico.	Media
Compatibilidad	3	La app y la web dependen del mismo contrato de la API y de Firebase; un contrato roto deja pantallas sin datos en ambos clientes.	Media
Capacidad de interacción	4	Cuatro roles con perfiles distintos, incluidos acudientes sin experiencia técnica y docentes que usan la app de pie en la puerta del aula.	Alta
Fiabilidad	5	La asistencia es un registro oficial: un escaneo perdido por falta de red es inaceptable. Por eso existe la cola de asistencia sin conexión.	Alta

Característica	Peso (0–5)	Justificación	Prioridad
Seguridad	5	Trata datos personales de menores (Ley 1581 de 2012), expone una API pública en Internet y gestiona autenticación con tokens JWT.	Alta
Mantenibilidad	4	Equipo de cinco aprendices que rota tareas por sprint; la arquitectura por capas debe conservarse para que cualquiera pueda modificar un módulo.	Alta
Flexibilidad	3	La app se distribuye como APK con actualización OTA; la API y la web se despliegan en Azure desde GitHub Actions.	Media
Protección (safety)	1	Una falla no causa daño físico ni ambiental. El riesgo de que un acudiente reciba tarde la información de llegada se cubre con Fiabilidad.	Baja

4. Especificación de métricas y umbrales

Para cada subcaracterística priorizada se define una métrica (basada en ISO/IEC 25023 o en la herramienta que la mide), la fórmula, el umbral de aceptación y el instrumento de medición.

Tabla 4. *Métricas, fórmulas y umbrales de aceptación*

Característica / Subcaracterística	Métrica	Fórmula	Umbral	Instrumento
Adecuación funcional / Completitud	Cobertura funcional (sistema)	HU implementadas y aceptadas ÷ HU comprometidas × 100	≥ 95 %	Backlog + PR-08
Adecuación funcional / Corrección	Tasa de casos aprobados (app, API, web)	Casos aprobados ÷ casos ejecutados × 100	≥ 95 %	flutter test · pytest · Vitest (CI), PR-05, PR-06
Eficiencia / Comportamiento temporal	Tiempo de arranque en frío (app)	Tiempo desde abrir la app hasta ver el login o el inicio del rol, en un Android de gama media	≤ 3 s	adb shell am start -W / Flutter DevTools (modo profile)
Eficiencia / Comportamiento temporal	Tiempo de respuesta p95 (API)	Percentil 95 del tiempo de respuesta de los endpoints que usan la app y la web	≤ 800 ms	k6
Eficiencia / Capacidad	Usuarios concurrentes (API)	Usuarios simultáneos con menos de 1 % de errores	≥ [50]	k6
Eficiencia / Utilización de recursos	Tamaño del APK (app)	Tamaño del APK de release para arm64-v8a	≤ [40] MB	flutter build apk --split-per-abi --analyze-size
Compatibilidad / Interoperabilidad	Contratos API verificados (app ↔ API)	Endpoints consumidos con prueba de mapeo DTO ÷ endpoints consumidos × 100	100 %	dto_mapping_test.dart · *_repository_test.dart + Swagger de la API

Característica / Subcaracterística	Métrica	Fórmula	Umbral	Instrumento
Capacidad de interacción / Inclusividad	Pautas de accesibilidad (app)	Pantallas que cumplen área táctil (Android e iOS), etiquetas y contraste WCAG AA ÷ pantallas evaluadas × 100	100 %	accessibility_test.dart + Accessibility Scanner
Capacidad de interacción / Inclusividad	Puntaje de accesibilidad (web)	Puntaje Lighthouse Accessibility de las páginas principales	≥ 90	Lighthouse / axe
Capacidad de interacción / Aprendizaje	Éxito de tareas (sistema)	Tareas completadas sin ayuda ÷ tareas asignadas × 100, por rol	≥ 85 %	PR-08 (UAT)
Capacidad de interacción / Protección contra errores de usuario	Entradas validadas (app, web)	Campos de formulario con validación en el cliente ÷ campos de formulario × 100	100 %	validators_test.dart · password.test.ts + revisión de código
Fiabilidad / Ausencia de fallos	Densidad de defectos (sistema)	Defectos confirmados ÷ KLOC	≤ 2	PR-07
Fiabilidad / Tolerancia a fallos	Asistencia sin conexión entregada (app)	Escaneos hechos sin red que llegan a la API sin pérdida ni duplicado ÷ escaneos × 100	100 %	attendance_queue_test.dart + PR-06 (modo avión)
Fiabilidad / Recuperabilidad	Degradación controlada (app)	Pantallas que muestran el error y permiten reintentar, sin cerrar la app, cuando la API falla ÷ pantallas con datos remotos × 100	100 %	student_home_resilience_test.dart + PR-06
Fiabilidad / Recuperabilidad	Tiempo de restauración (BD)	Tiempo real de restauración de PostgreSQL en simulacro	≤ [RTO]	TR-01
Seguridad / Confidencialidad	Vulnerabilidades (app, API, web)	Vulnerabilidades críticas + altas abiertas en el código y sus dependencias	0	MobSF + Dependabot (app) · pip-audit (API) · npm audit (web)
Seguridad / Confidencialidad	Almacenamiento seguro (app)	Tokens o credenciales guardados fuera del almacenamiento cifrado (Keystore)	0	Revisión de código + MobSF
Seguridad / Autenticidad	Endpoints protegidos (API)	Endpoints con autenticación ÷ endpoints no públicos × 100	100 %	test_authorization.py (CI) + revisión
Mantenibilidad / Analizabilidad	Hallazgos de análisis estático (app, API, web)	Errores reportados por el analizador de cada componente	0	flutter analyze · ruff · ESLint + tsc (CI)
Mantenibilidad / Modularidad	Violaciones de arquitectura (app)	Imports que cruzan una frontera de capa prohibida	0	architecture_test.dart

Característica / Subcaracterística	Métrica	Fórmula	Umbral	Instrumento
Mantenibilidad / Modificabilidad	Duplicación (sistema)	Líneas duplicadas ÷ líneas totales	≤ 3 %	SonarQube Cloud o jscpd
Mantenibilidad / Capacidad de ser probado	Cobertura de la lógica (app)	Líneas cubiertas ÷ líneas ejecutables en las capas domain y data	≥ 60 % al liberar (piso CI: 75 %)	flutter test --coverage + tool/coverage_report.dart
Mantenibilidad / Capacidad de ser probado	Cobertura de líneas (API)	Líneas cubiertas ÷ líneas ejecutables del paquete app/	≥ 60 %	pytest --cov (CI)
Flexibilidad / Instalabilidad	Instalación exitosa (sistema)	Instalaciones exitosas siguiendo DS-04 ÷ intentos × 100	100 %	DS-04
Flexibilidad / Reemplazabilidad	Actualización OTA exitosa (app)	Dispositivos que pasan a la última release desde el aviso de la app ÷ intentos × 100	100 %	UpdateService + GitHub Releases

Justificación de los umbrales e instrumentos ajustados

La plantilla propone instrumentos pensados para aplicaciones web. Se conservan donde aplican (panel web y API) y se ajustan para la app móvil con esta justificación:

• Accesibilidad: Lighthouse se usa en el panel web. En la app Flutter, que no es una página web, se usan las pautas que trae Flutter (área táctil, etiquetas y contraste WCAG AA) en accessibility_test.dart, más Accessibility Scanner en el dispositivo.

• Seguridad: cada componente audita sus dependencias en su CI (pip-audit en la API, npm audit en la web, Dependabot en la app). El APK se revisa con MobSF guiado por OWASP MASVS. OWASP ZAP queda como escaneo dinámico de la API y la web antes de liberar (QG5).

• Analizabilidad: el rating SQALE requiere un servidor SonarQube. Se mide con el analizador de cada componente (flutter analyze, ruff, ESLint y tsc), que corren en cada pull request. Si el equipo habilita SonarQube Cloud, su rating se agrega como métrica adicional.

• Cobertura: medir solo el código nuevo exige SonarQube. En la app se mide la lógica (domain y data), porque las pantallas se validan con pruebas de sistema (PR-06) y de aceptación (PR-08): la meta es 60 %, la línea base 81,5 % y el CI exige un piso de 75 %. En la API se mide la cobertura de líneas (línea base 61 %). La web tiene pruebas de sus reglas de permisos, guardia de rutas y cliente de API; su cobertura se agregará como métrica cuando las pruebas cubran los hooks.

Tabla 4a. *Línea base medida el 25/10/2026 (rama chore/modelo-calidad de la app, chore/calidad-seguridad de la API y de la web)*

App — análisis y pruebas	flutter analyze: 0 hallazgos · 110 de 110 pruebas aprobadas — cumple
App — cobertura de la lógica	81,5 % (domain 69,8 %, data 85,1 %) — cumple la meta de 60 %
App — arquitectura	0 violaciones de capas — cumple
App — accesibilidad	2 de 2 pantallas evaluadas — cumple, después de etiquetar 14 botones de ícono y subir el contraste del texto secundario de 3,95:1 a 5,0:1

App — seguridad y tamaño	Tokens en flutter_secure_storage (Keystore) · APK arm64 de 27,6 MB — cumple
API — análisis y pruebas	ruff: 0 hallazgos (164 líneas largas excluidas) · 26 de 26 pruebas aprobadas · cobertura de líneas 61 % — cumple
API — dependencias	pip-audit: 0 vulnerabilidades (antes 16 en 7 paquetes) — cumple
API — endpoints protegidos	94 de 94 endpoints no públicos exigen token; 10 públicos declarados (autenticación y catálogos del registro) — cumple
Web — análisis y pruebas	ESLint: 0 errores (28 advertencias) · tsc: 0 errores · 28 de 28 pruebas aprobadas · build de producción — cumple
Web — dependencias	npm audit: 0 vulnerabilidades (antes 9: 1 crítica en Next.js, 4 altas) — cumple
Tamaño del código	App 18,7 KLOC · API 4,2 KLOC · Web 20,7 KLOC (base para la densidad de defectos)

5. Modelo de evaluación y agregación

Cada métrica se normaliza a una escala 0–100 % de cumplimiento respecto a su umbral. El puntaje de una característica es el promedio de sus métricas; el **índice global de calidad (IGC)** es el promedio ponderado por los pesos de la Tabla 3:

```
Fórmula
IGC = Σ (peso_i × puntaje_i) / Σ peso_i

Nivel de calidad:  IGC ≥ 90 % → Excelente   |  80–89 % → Satisfactorio
                   70–79 % → Aceptable con plan de mejora   |  < 70 % → Insatisfactorio
```

Normalización: en métricas «mayor o igual» el cumplimiento es min(100, valor ÷ umbral × 100); en métricas «menor o igual» es 100 si el valor no supera el umbral y umbral ÷ valor × 100 si lo supera; en métricas de cero o 100 % el cumplimiento es 100 o 0. Protección (safety) no tiene métricas y su peso no entra en el cálculo, por lo que Σ peso_i = 32.

6. Proceso de evaluación (ISO/IEC 25040) y puntos de control

Tabla 5. *Proceso de evaluación de la calidad*

Etapa ISO/IEC 25040	Actividad en el proyecto	Producto
1. Establecer requisitos de evaluación	Definir propósito, alcance y características priorizadas	Secciones 1–3
2. Especificar la evaluación	Seleccionar métricas y umbrales	Sección 4
3. Diseñar la evaluación	Planificar cuándo, cómo y con qué herramienta se mide	MC-02, PR-01
4. Ejecutar la evaluación	Medir y registrar resultados	PR-03 a PR-08
5. Concluir la evaluación	Comparar contra umbrales y decidir	PR-09, CI-01

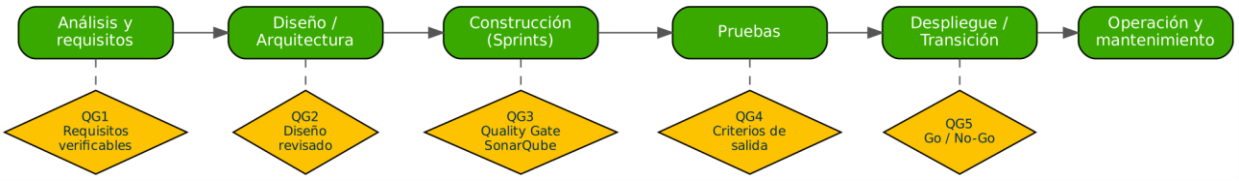


Figura 2. *Puntos de control de calidad (quality gates) en el ciclo de vida* Fuente: Elaboración propia con base en ISO/IEC/IEEE 12207

Tabla 6. *Definición de los quality gates*

Quality gate	Momento	Criterio para superarlo	Responsable
QG1	Cierre de análisis	HU con criterios de aceptación verificables (INVEST)	[Product Owner]
QG2	Cierre de diseño	Arquitectura y modelo de datos revisados	[Líder técnico]
QG3	Cada pull request	CI en verde en el repositorio del PR: app (flutter analyze, pruebas y cobertura de la lógica ≥ 75 %), API (ruff, pytest y pip-audit) y web (ESLint, tsc, Vitest, build y npm audit)	GitHub Actions (ci.yml)
QG4	Cierre de pruebas	Criterios de salida del PR-01 cumplidos y cobertura de la lógica de la app ≥ 60 %	[Líder de pruebas]
QG5	Antes de liberar	Decisión go / no-go en CI-01	[Comité: PO + instructor]

7. Roles y responsabilidades de calidad

Rol	Responsabilidades
Product Owner	Prioriza características, valida criterios de aceptación, firma el acta de aceptación.
Líder de calidad / pruebas	Mantiene este modelo, consolida métricas, emite PR-09 y propone la decisión de liberación.
Desarrolladores	Escriben pruebas (Flutter, pytest y Vitest), corrigen los hallazgos del análisis estático y los defectos asignados.
Scrum Master	Garantiza que la Definición de Terminado incluya los criterios de calidad.
Instructor	Valida umbrales, revisa evidencias y aprueba el paquete documental.

8. Definición de Terminado (DoD) alineada al modelo

- ☐ El código compila y el workflow CI (ci.yml) del repositorio termina en verde.
- ☐ Pruebas escritas y aprobadas; la cobertura no baja del piso del CI.
- ☐ Análisis estático sin errores y 0 vulnerabilidades críticas o altas en las dependencias.
- ☐ Revisión de código aprobada por al menos un compañero.
- ☐ Criterios de aceptación de la HU verificados.
- ☐ Documentación técnica (README) y de API actualizada.

- ☐ Commit y PR siguen las convenciones de GC-01.
- ☐ Pantallas nuevas accesibles: accessibility_test (app) o Lighthouse ≥ 90 (web).

9. Control de cambios del modelo

Cualquier cambio en pesos, métricas o umbrales se registra en el historial de versiones con su justificación y debe ser aprobado por el Product Owner y el instructor antes de aplicarse a una evaluación.

10. Firmas

[Nombre completo]	[Nombre completo]	[Nombre completo]
Elaboró	**Revisó**	**Aprobó**
Fecha: [dd/mm/aaaa]	Fecha: [dd/mm/aaaa]	Fecha: [dd/mm/aaaa]